

CIFAR-10 训练 ResNet 实验报告

1. 实验概要

实验目的： 实验以 CIFAR-10 图像分类为任务，先建立小型的 CNN Baseline，再建立适用于 32×32 图像的 ResNet18。通过实验，复习并强化前四周关于神经网络学到的概念与知识，主要实现以下目标：

1. 深入理解神经网络基本架构与对应的代码实现，理解 CNN 以及 ResNet 的残差连接块。
2. 加强对数据集的认识与理解，明确 train、validation 与 test 三种 dataset 是如何划分的、如何处理的（如数据增强只能应用在 train 数据集中），理解三种数据集的作用（train 用于更新模型参数，validation 用于选择模型超参数，test 用于评估性能）。
3. 将训练神经网络的理论知识应用在实践中，从网络架构选择、参数选择、超参数优化到最终模型的性能测试与评估，熟悉完整的训练与推理过程，理解 loss，accuracy 等训练指标，应用 dropout、数据增强等正则化技术。

实验内容： 在 2.1 小节中先用小 CNN 建立 Baseline 模型，在 2.2 小节中实现 CIFAR-10 Style 的 ResNet18，在第 3 节中通过 Dropout, Data Augmentation 与 Width Reduction 三种方法探索模型的改进，第 4 节展示 Baseline、ResNet 及消融实验的具体实验配置，第 5 节从“CNN Baseline vs ResNet”与“ResNet 的 3 种优化方法效果”两个角度进行实验结果分析。

实验流程： `dataset.py` 负责划分和处理数据集，`model.py` 定义网络结构，`train.py` 完成模型训练与评估。每个 epoch 在 validation 上计算指标并保存 validation accuracy 最高的 checkpoint，训练结束后加载该 checkpoint，在 test 上进行一次最终评估，最后根据实验记录比较模型性能。

2.模型架构

2.1 CNN Baseline

CNN Baseline 由三层 kernel size 为 3 的卷积组成，输出通道数依次为 32、64 和 128。每层卷积后用 ReLU 引入非线性，并通过两次 MaxPool 将特征图尺寸从 32×32 逐步降至 8×8 ，以减少后续计算量。最后使用全局平均池化将每个通道压缩为一个特征，再由线性分类器输出 10 个类别的 logits。

各 stage 尺寸表如下：

阶段	主要操作	输出尺寸 ($C \times H \times W$)
输入	CIFAR-10 图像	$3 \times 32 \times 32$
Layer 1	3×3 Conv, $3 \rightarrow 32$ + ReLU	$32 \times 32 \times 32$
Layer 2	3×3 Conv, $32 \rightarrow 64$ + ReLU + 2×2 MaxPool	$64 \times 16 \times 16$
Layer 3	3×3 Conv, $64 \rightarrow 128$ + ReLU + 2×2 MaxPool	$128 \times 8 \times 8$
Pool	Global Average Pooling	$128 \times 1 \times 1$
Classifier	Flatten; Linear, $128 \rightarrow 10$	10 个 logits

Baseline 模型共有 9.4538 万个参数，作为后续 ResNet18 模型的参照。

2.2 CIFAR-10 Style ResNet18

首先定义 Basic Residual Block 类 `BasicBlock`。它与 UDL 图 11.7(a) 的基本结构相同，都使用两个 3×3 卷积学习残差函数 $F(x)$ ，再与 shortcut 相加得到 $y = F(x) + x$ ；区别是我用了 ResNet18 中更常用的 post-activation 顺序。BasicBlock 中的残差连接让每个 block 学习对已有表示的修正，让梯度的传播路径更为直接。

ResNet18 共包含 4 个 stage，每个 stage 有 2 个 BasicBlock，即 $[2, 2, 2, 2]$ 。因为 CIFAR-10 的输入只有 32×32 ，所以 stem 用了 3×3 、stride=1 的卷积。Layer 2~4 的第一个 block 用 stride=2 完成下采样（特征图尺寸变小），并通过 1×1 shortcut 匹配特征图尺寸和通道数。

各 stage 尺寸表如下：

阶段	主要操作	输出尺寸 ($C \times H \times W$)
输入	CIFAR-10 图像	$3 \times 32 \times 32$
Stem	3×3 Conv, $3 \rightarrow 64$ + BN + ReLU	$64 \times 32 \times 32$
Layer 1	2 个 BasicBlock, stride=1	$64 \times 32 \times 32$
Layer 2	2 个 BasicBlock, 第一个 block stride=2	$128 \times 16 \times 16$
Layer 3	2 个 BasicBlock, 第一个 block stride=2	$256 \times 8 \times 8$
Layer 4	2 个 BasicBlock, 第一个 block stride=2	$512 \times 4 \times 4$
Pool	Global Average Pooling	$512 \times 1 \times 1$
Classifier	Flatten; Linear, $512 \rightarrow 10$	10 个 logits

基础的 width64 ResNet18 模型共有 1117.3962 万个参数，约为 CNN Baseline 的 118 倍。

3.模型改进方案

3.1 Dropout

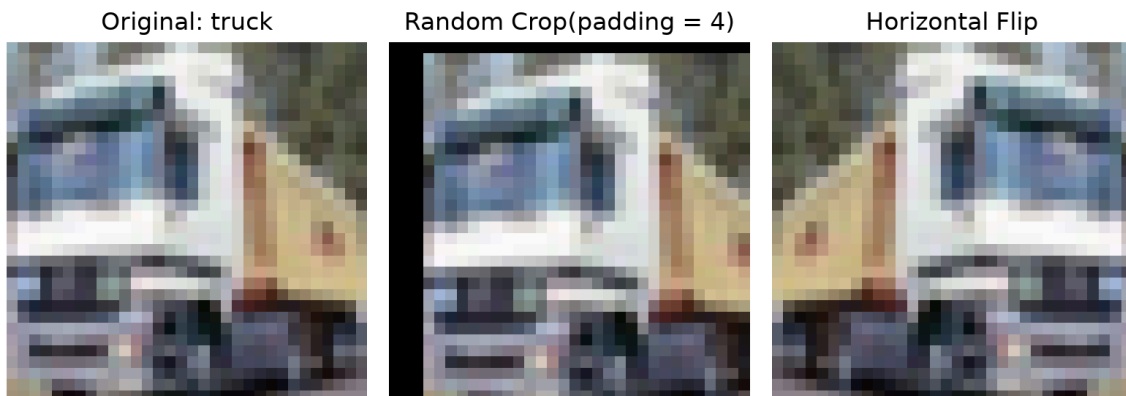
Dropout 是一种训练神经网络模型时常用的正则化技术，通过在训练时随机将部分特征置零，降低模型对特定特征的依赖，从而减轻过拟合。

本实验在 ResNet18 尾部的线性 Classifier 前引入 $p=0.3$ 的 Dropout，直接作用于 AvgPool 后的特征向量，并不改变卷积特征图。Dropout 不改变模型参数，不一定会提高 Accuracy，实际效果要通过实验判断。

3.2 Data Augmentation

Data Augmentation 是另一种正则化技术，Dropout 是针对模型的正则化技术，而数据增强是针对数据集的正则化技术，在训练数据集有限时，通过合理的变换增加训练数据的多样性，减少模型对于特定图像模式的依赖，可能提高模型的泛化能力。

本实验对 train 数据集的图像先进行 padding=4，再随机裁剪回 32×32 ，并以 0.5 的概率进行水平翻转。Validation 和 test 的图像不使用随机增强，只进行 ToTensor 和 Normalize，保证评估结果稳定且可比较。



3.3 Width Reduction

模型超参数的选择对其性能也有着重要的影响。在本实验的 ResNet18 模型中，网络宽度(base channels) 决定每个 stage 的特征通道数，是控制模型容量和计算成本的重要超参数。

本实验将基础通道数从 64 降为 32，使 4 个 stage 的通道数从 64/128/256/512 变成了 32/64/128/256。因为卷积层的参数量近似为 $k^2 C_{in} C_{out}$ ，所以卷积层的总参数量降为原先的约 1/4（实际模型参数量：1117.3962 万 -> 279.7610 万），可以减少训练时间和训练时的内存占用，但是可能带来 Accuracy 等指标的退化。

4.实验设置

4.1 数据集划分

CIFAR-10 包含 60,000 张 32×32 彩色图像，共有 10 个类别。实验将官方 50,000 张训练图像按固定随机种子划分为 45,000 张 train 和 5,000 张 validation，并保留官方 10,000 张 test 图像。Train 用于计算梯度和更新参数；validation 用于选择最佳 epoch；test 不参与训练和模型选择，只在训练结束后评估一次。

所有图像都先转换为 Tensor，再使用 CIFAR-10 的通道均值和标准差进行 Normalize。只有启用 Data Augmentation 的实验会对 train 图像进行随机裁剪和水平翻转，validation 和 test 始终使用固定的数据处理流程。

4.2 训练参数

为了让不同模型和改进方案可以直接比较，基本的训练设置保持一致，如下：

设置	参数
Random seed	266978
Epoch / Batch size	200 / 128
Loss	CrossEntropyLoss
Optimizer	SGD, lr=0.1, momentum=0.9
Weight decay	0.0005
LR scheduler	CosineAnnealingLR, 最终下降至 0
Device	Apple MPS
最佳模型	validation accuracy 最高的 checkpoint
最终测试	加载最佳 checkpoint 后测试一次

说明：

1. Cross-entropy 不只判断分类是否正确，还会考虑模型对正确类别的置信度，而 Accuracy 只判断最大 logit 对应的类别，所以要同时观察 Loss 与 Accuracy。
2. SGD 根据 mini-batch 梯度更新参数，Momentum 保留一部分历史更新方向。
3. CosineAnnealingLR 则让前期更新较快、后期逐渐减小学习率。

4.3 消融实验设计

消融实验只改变 Dropout、Crop/Flip 和 Width32 三个因素。三个因素各有开和关两种状态，共 $2 \times 2 \times 2 = 8$ 组实验。完整 8 种组合的实验结果可以分别判断每种改进的作用，而不是只比较原始模型和最终模型。

实验	Dropout	Crop/Flip	Base channels
Reference	✗	✗	64
+ Dropout	✓	✗	64
+ Crop/Flip	✗	✓	64
+ Width32	✗	✗	32
+ Dropout + Crop/Flip	✓	✓	64
+ Dropout + Width32	✓	✗	32
+ Crop/Flip + Width32	✗	✓	32
三种全部加入	✓	✓	32

5.实验结果与分析

5.1 Baseline & ResNet18(Origin)

首先在不使用 Dropout 和 Data Augmentation 的条件下比较 CNN Baseline 与 width64 ResNet18。两组实验使用相同的数据划分、训练参数和随机种子。

模型	最佳 epoch	Val Accuracy	Test Accuracy	Test Loss	参数量 (万)	训练时间	平均每个 epoch	MPS 已分配内存
CNN Baseline	185	80.06%	80.10%	0.6555	9.4538	0.32 h	5.74 s	1.45 MiB
ResNet18	135	89.08%	88.99%	0.3720	1117.3962	2.33 h	41.75 s	193.81 MiB

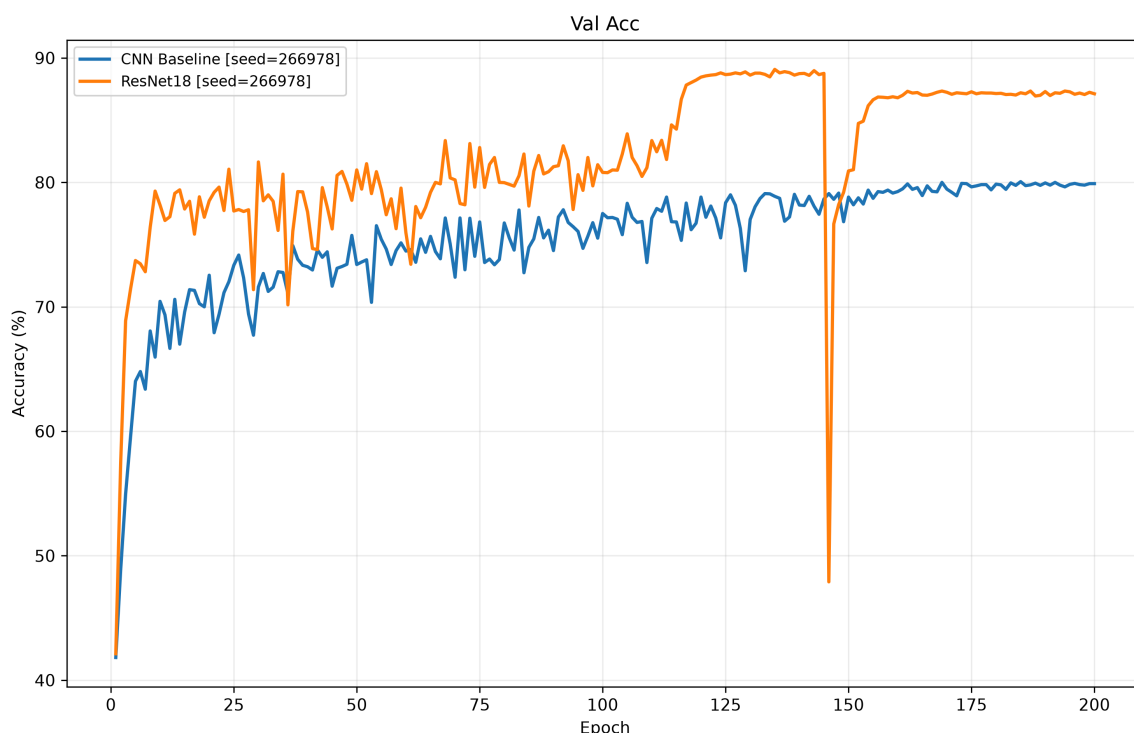


图 5-1: CNN Baseline 与 ResNet18 的 validation accuracy。

ResNet18 的 Test Accuracy 比 Baseline 高 8.89%，Test Loss 从 0.6555 降至 0.3720，说明更深的残差网络不仅分类正确率更高，预测结果的整体置信度也更合理。主要原因是 ResNet18 有更大的模型容量，并通过残差连接减小了深层网络的优化难度，能够学习更复杂的图像特征。

但是上述的指标提升背后是训练成本的增加。ResNet18 的参数量约为 Baseline 的 **118 倍**，训练时间约为 **7.3 倍**，记录的 MPS 已分配内存约为 **134 倍**。综合考虑，Baseline 更适合快速检查数据和训练流程，ResNet18 的优势是更高的分类性能。

5.2 三种优化方法的消融实验

八组实验都训练 200 个 epoch，并使用各自 validation accuracy 最高的 checkpoint 进行最终测试。完整结果如下：

实验	Val Accuracy	Test Accuracy	Test Loss	参数量	训练时间	MPS 已分配内存
Reference	89.08%	88.99%	0.3720	11.174 M	2.23 h	193.81 MiB
+ Dropout	88.80%	89.26%	0.3571	11.174 M	2.28 h	193.93 MiB
+ Crop/Flip	95.18%	95.12%	0.1834	11.174 M	2.26 h	193.81 MiB
+ Width32	87.70%	88.15%	0.4005	2.798 M	0.76 h	50.92 MiB
+ Dropout + Crop/Flip	95.24%	94.96%	0.1983	11.174 M	2.20 h	193.93 MiB
+ Dropout + Width32	88.44%	87.98%	0.4072	2.798 M	0.77 h	50.92 MiB
+ Crop/Flip + Width32	94.64%	94.13%	0.2253	2.798 M	0.82 h	50.92 MiB
三种全部加入	94.44%	94.26%	0.2343	2.798 M	0.83 h	50.92 MiB

5.2.1 Accuracy 与 Loss

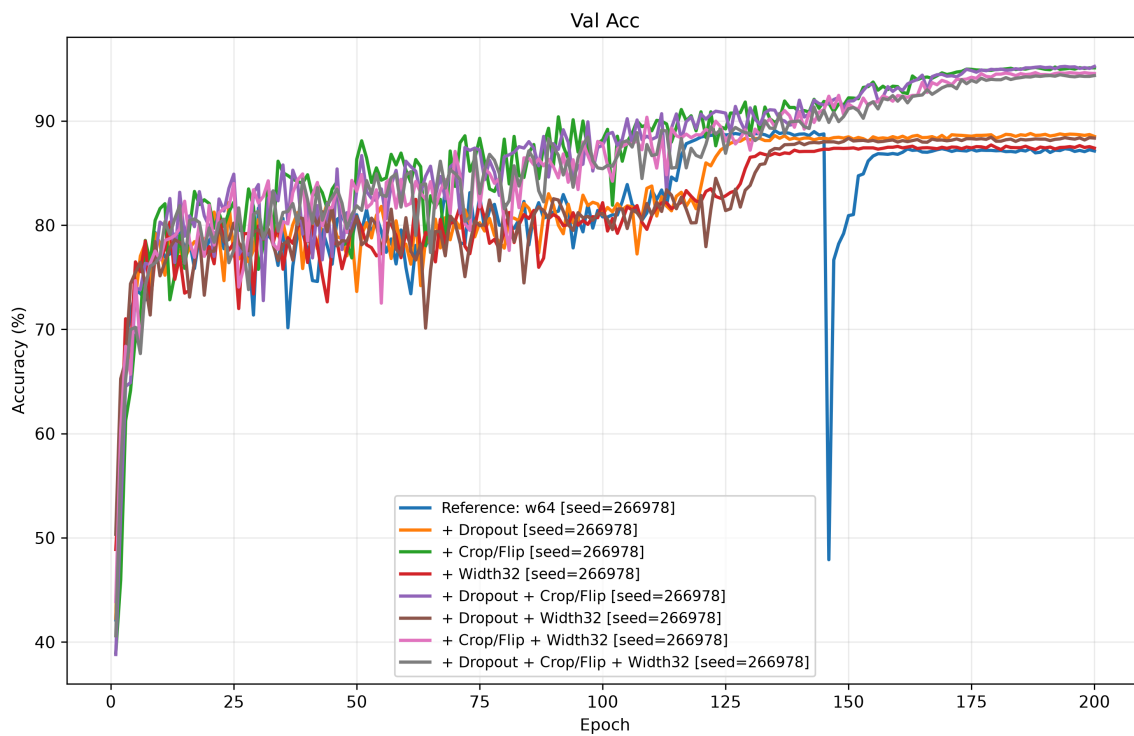


图 5-2: 八组消融实验的 validation accuracy。

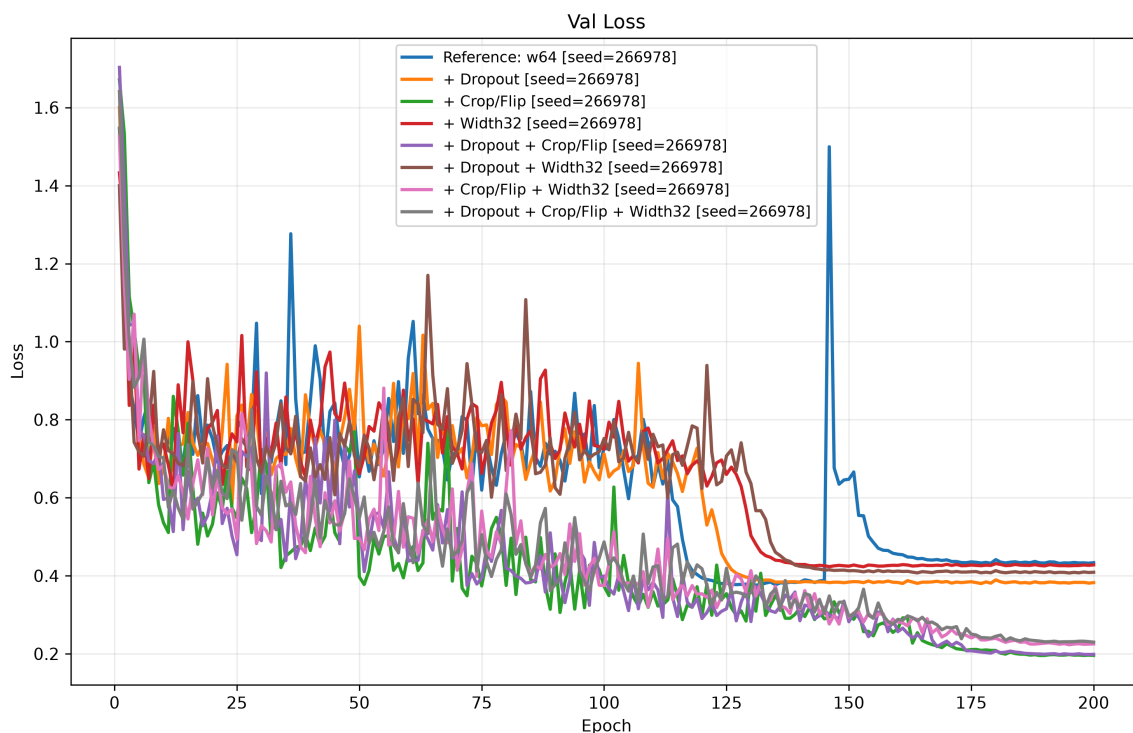


图 5-3: 八组消融实验的 validation loss。

Crop/Flip 是 Accuracy 提升的主要来源。在八组完整组合中，对是否启用 Crop/Flip 分别取平均，启用后 Test Accuracy 平均提高 6.02%，而且不增加参数量。随机裁剪和水平翻转增加了训练图像的空间变化，使模型不能只记住物体的固定位置和方向，所以 validation loss 更低，validation accuracy 也明显更高。

Dropout 没有获得稳定的收益。单独加入 Dropout 后 Test Accuracy 提高了 0.27%，但在完整八组实验中，Dropout 的平均影响只有约 +0.02%；加入 Crop/Flip 后，Dropout 也没有继续提高 Test Accuracy。它只作用于 AvgPool 后的特征向量，而卷积主体已经使用 BatchNorm 和 weight decay，所以这种 Dropout 带来的额外正则化较弱。

Width32 的 Test Accuracy 平均下降约 0.95%，说明缩小模型容量会损失一部分特征表达能力。但加入 Crop/Flip 后，width32 仍达到 94.13% Test Accuracy，比 width64 Reference 高 5.14%。这说明对于 ResNet18 下的 CIFAR-10 的图像分类任务，训练数据的多样性比继续增加网络宽度更重要。

另外，注意到图中蓝色的 Reference 的 validation accuracy 在 epoch 146 从 88.76% 突然降至 47.90%，训练 accuracy 也同时下降，这是意料之外的异常。分析发现，当时学习率仍按 cosine 曲线平滑变化，并不是 scheduler 引起的。更可能是某个 mini-batch 产生了较大的参数更新，Momentum 延续了该方向，同时影响了 BatchNorm 的统计量。因为最佳的 checkpoint 保存在 epoch 135，所以最终 Test Accuracy 没有使用训练后期退化的模型，并不影响最终实验结果的对比分析。

5.2.2 Time、Parameters 与 Memory

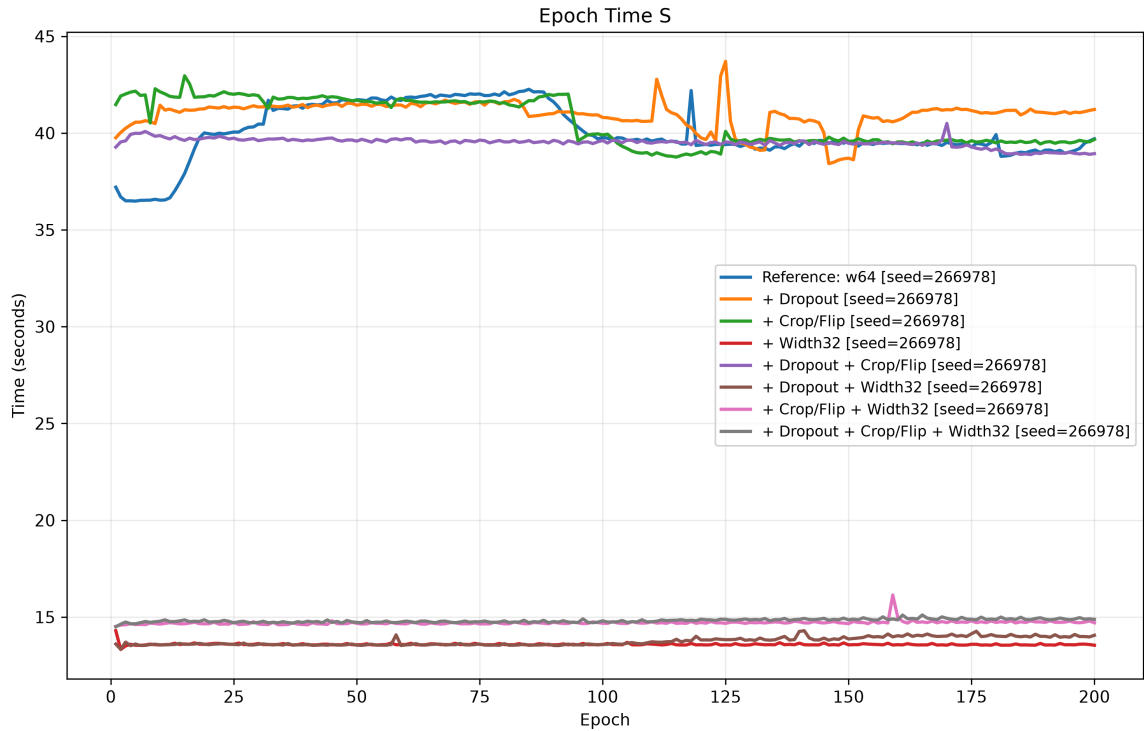


图 5-4: 不同网络宽度下的单个 epoch 训练时间。

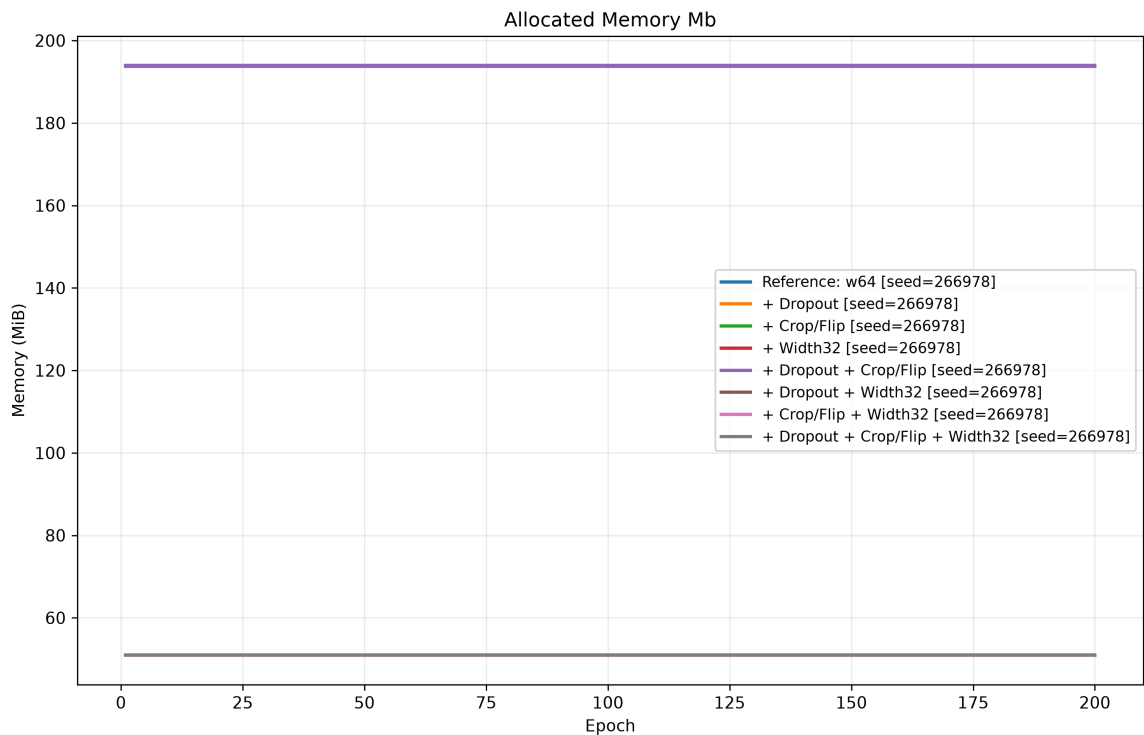


图 5-5: 不同网络宽度下 PyTorch 记录的 MPS 已分配内存。

Width32 是降低训练成本最有效的改动。与 width64 的组合平均值相比，它减少了 74.96% 的参数量、64.67% 的训练时间和 73.73% 的 MPS 已分配内存。原因是大部分卷积层的输入和输出通道同时减半，参数量和主要计算量接近原来的四分之一。

如果只看最高 Test Accuracy，width64 + Crop/Flip 达到 95.12%；如果同时考虑 Accuracy、Time、Parameters 和 Memory，width32 + Crop/Flip 更合适，它的 Test Accuracy 为 94.13%，但只保留约 25.04% 的参数量、36.8% 的训练时间和 26.3% 的已分配内存。三种方法全部加入后 Test Accuracy 只比它高 0.13 个百分点，而 validation accuracy 和 Test Loss 更差，Dropout 似乎并不值得保留。

6.实验总结

本实验从小型 CNN Baseline 开始，实现了适用于 CIFAR-10 的 ResNet18，并通过 8 组消融实验分析 Dropout、Data Augmentation 和 Width Reduction 的作用。实验中 train 用于更新参数，validation 用于选择最佳 checkpoint，test 只用于最终评估，避免测试集参与模型选择。

实验得到以下结论：

1. ResNet18 通过更大的模型容量和残差连接，将 Test Accuracy 从 80.10% 提高到 88.99%，但同时显著增加参数量、训练时间和内存占用。
2. Crop/Flip 是本实验最有效的改进，平均提高约 6.02% 的 Test Accuracy，说明合理的数据增强比单纯增加模型容量更有效。
3. Width32 用约 0.95% 的平均 Accuracy 损失，换来约 75% 的参数量下降和明显的时间、内存节省，是最有效的效率改进。
4. Dropout 在本实验的目标任务中没有稳定的收益。正则化是否有效取决于模型结构、使用位置和已有训练策略，不能只根据单组实验判断。

综上所述，仅考虑最高 Accuracy，可以选择 width64 + Crop/Flip；而如果希望在 Accuracy 和训练成本之间取得更好的平衡，则选择 width32 + Crop/Flip。

注：

1. 本实验只使用了一个随机种子，所以小于 0.3 个百分点的差异并不是稳定的结论。
2. MPS 只记录了 PyTorch 当前已分配的内存，不是整机内存或严格的峰值内存，结论仅供参考。
3. 实验并未测量推理延迟和吞吐量，要搭建性能更平衡的网络需要进一步的实验探索。